6901012610196 Chonpatchut Boonpalit Cpr.E S.2

เนื่องจากหนูไม่พบ error ในระหว่างเขียน หนูจึงเขียนเป็น error ที่อาจเกิดขึ้นแทนนะคะ

Part 1: Graphic Mode

Exercise 1.1 — Balloon Movement

โจทย์ให้ทำอะไร

ให้สร้างโปรแกรมกราฟิกที่วาดวงกลมแทนลูกโป่ง และให้ลูกโป่งเคลื่อนที่จากซ้ายไปขวา เมื่อเคลื่อนที่จนถึงตำแหน่งที่กำหนด ให้กลับไปเริ่มต้นที่ตำแหน่งเดิมอีกครั้ง

Code

```
x = 100


def setup():
    size(600, 400)


def draw():
    background(255)

    ellipse(x, 200, 50, 50)

    move_balloon()


def move_balloon():
    global x

    x = x + 10

    if x > 500:
        x = 100
```

อธิบาย Code

บรรทัดที่ 1

```
x = 100
```

กำหนดค่าเริ่มต้นของตำแหน่ง x ให้เท่ากับ 100

ฟังก์ชัน `setup()`

```
def setup():
    size(600, 400)
```

สร้างหน้าต่างขนาดกว้าง 600 และสูง 400 pixels

ฟังก์ชัน `draw()`

```
def draw():
```

เป็นฟังก์ชันที่ Processing เรียกซ้ำเพื่อสร้าง animation

```
background(255)
```

ล้างหน้าจอและกำหนดพื้นหลังเป็นสีขาว

```
ellipse(x, 200, 50, 50)
```

วาดวงกลมที่ตำแหน่ง `x, 200` ขนาด 50 × 50

```
move_balloon()
```

เรียกฟังก์ชันสำหรับเปลี่ยนตำแหน่งลูกโป่ง

ฟังก์ชัน `move_balloon()`

```
global x
```

บอกว่าฟังก์ชันนี้ต้องการแก้ไขตัวแปร `x` ที่อยู่นอกฟังก์ชัน

```
x = x + 10
```

เพิ่มตำแหน่ง `x` ทีละ 10 pixels ทำให้ลูกโป่งเคลื่อนที่ไปทางขวา

```
if x > 500:
    x = 100
```

ถ้า `x` มากกว่า 500 ให้กลับไปที่ 100

Pseudocode

START

Set x = 100

Create window 600 x 400

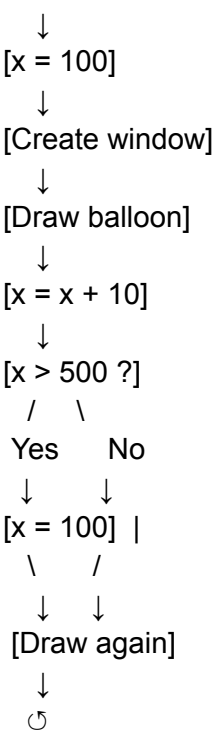
```
REPEAT
   Clear background
   Draw balloon at (x, 200)

   x = x + 10

   IF x > 500
      x = 100
   END IF
END REPEAT

END
```

Flowchart



Error ที่พบ

Error: ลูกโป่งไม่เคลื่อนที่

สาเหตุคือไม่ได้เรียก `move_balloon()` ใน `draw()` หรือไม่ได้เพิ่มค่า `x`

วิธีแก้:

move_balloon()

และ

x = x + 10

ต้องทำงานทุก frame

Exercise 1.2 — Random Movement

โจทย์ให้ทำอะไร

ให้สร้างวงกลมที่เกิดขึ้นในตำแหน่งแบบสุ่ม และกำหนดขนาดของวงกลมแบบสุ่มเช่นกัน

Code

```
def setup():
    size(500, 500)


def draw():
    background(255)

    x = random(100, 400)
    y = random(150, 350)

    draw_circle(x, y, random(3, 30))


def draw_circle(x, y, r):
    ellipse(x, y, r, r)
```

อธิบาย

def setup():

สร้างฟังก์ชันสำหรับตั้งค่าก่อนเริ่มโปรแกรม

size(500, 500)

สร้างหน้าต่างขนาด 500 × 500

background(255)

ล้างหน้าจอและทำพื้นหลังสีขาว

x = random(100, 400)

สุ่มตำแหน่ง x

y = random(150, 350)

สุ่มตำแหน่ง `y`

draw_circle(x, y, random(3, 30))

เรียกฟังก์ชัน `draw_circle()` พร้อมส่งตำแหน่งและขนาดแบบสุ่ม

def draw_circle(x, y, r):

สร้างฟังก์ชันสำหรับวาดวงกลม

ellipse(x, y, r, r)

วาดวงกลมตามตำแหน่งและขนาดที่ได้รับ

Pseudocode

START

Create window 500 x 500

REPEAT
  Clear background

  Random x from 100 to 400
  Random y from 150 to 350
  Random radius from 3 to 30

  Draw circle
END REPEAT

END

Flowchart

[Start]
  ↓
[Create window]
  ↓
[Clear background]
  ↓
[Random x]
  ↓
[Random y]
  ↓
[Random size]
  ↓
[Draw circle]
  ↓
[Repeat]
  ↺

Error ที่พบ

ถ้าเขียนชื่อฟังก์ชันไม่ตรงกัน เช่น

drawCircle(...)

แทน

draw_circle(...)

จะทำให้โปรแกรมหา function ไม่เจอ

วิธีแก้: ชื่อฟังก์ชันที่เรียกต้องตรงกับชื่อที่ประกาศไว้

Exercise 1.3 — Distance with Line

โจทย์ให้ทำอะไร

ให้คำนวณระยะห่างระหว่างตำแหน่งของลูกโป่งกับตำแหน่งเมาส์ แล้วแสดงระยะดังกล่าวด้วยเส้น

Code

```
x = 200


def setup():
    size(600, 400)


def draw():
    background(255)

    draw_balloon(x, 200, 50)

    d = cal_distance(x, 200, mouseX, mouseY)

    line(x, 200, x + d, 200)


def draw_balloon(x, y, size):
    ellipse(x, y, size, size)


def cal_distance(x1, y1, x2, y2):
    diffx = abs(x1 - x2)
    diffy = abs(y1 - y2)
```

```
    result = sqrt(diffx * diffx + diffy * diffy)

    return result
```

อธิบายส่วนสำคัญ

x = 200

กำหนดตำแหน่งเริ่มต้นของลูกโป่ง

draw_balloon(x, 200, 50)

วาดลูกโป่งที่ `(x, 200)`

d = cal_distance(x, 200, mouseX, mouseY)

เรียก `cal_distance()` เพื่อหาระยะห่างระหว่างลูกโป่งกับเมาส์ แล้วเก็บไว้ใน `d`

diffx = abs(x1 - x2)

หาระยะห่างในแนวนอน

diffy = abs(y1 - y2)

หาระยะห่างในแนวตั้ง

result = sqrt(diffx * diffx + diffy * diffy)

ใช้สูตรระยะห่างระหว่างสองจุด

return result

ส่งค่าระยะกลับไปยังจุดที่เรียก function

line(x, 200, x + d, 200)

วาดเส้นที่มีความยาว `d`

Pseudocode

```
START

Set x = 200
Create window

REPEAT
    Clear background
    Draw balloon

    Calculate distance between balloon and mouse
```

    Store distance in d

    Draw line with length d
END REPEAT

END

Flowchart

[Start]
  ↓
[x = 200]
  ↓
[Create window]
  ↓
[Draw balloon]
  ↓
[Calculate distance]
  ↓
[d = distance]
  ↓
[Draw line]
  ↓
[Repeat]
  ↺

Error ที่พบ

ถ้าลืม `return result`:

return result

ตัวแปร `d` จะไม่ได้รับค่าระยะทางที่ต้องการ

Exercise 1.4 — Follow Mouse Direction

โจทย์ให้ทำอะไร

ให้โปรแกรมตรวจสอบว่าตำแหน่งเมาส์อยู่ด้านซ้ายหรือด้านขวาของลูกโป่ง แล้ววาดเส้นไปทางเดียวกับเมาส์

Code

x = 300

```
def setup():
    size(600, 400)


def draw():
    background(255)

    draw_balloon(x, 200, 50)

    d = cal_distance(x, 200, mouseX, mouseY)

    if mouseX > x:
        line(x, 200, x + d, 200)
    else:
        line(x, 200, x - d, 200)


def draw_balloon(x, y, size):
    ellipse(x, y, size, size)


def cal_distance(x1, y1, x2, y2):
    diffx = abs(x1 - x2)
    diffy = abs(y1 - y2)

    result = sqrt(diffx * diffx + diffy * diffy)

    return result
```

อธิบาย

if mouseX > x:

ตรวจสอบว่าเมาส์อยู่ทางขวาของลูกโป่งหรือไม่

ถ้าใช่:

line(x, 200, x + d, 200)

วาดเส้นไปทางขวา

ถ้าไม่ใช่:

else:
    line(x, 200, x - d, 200)

วาดเส้นไปทางซ้าย

Pseudocode

```
START

Set x = 300

Create window

REPEAT
   Clear background
   Draw balloon

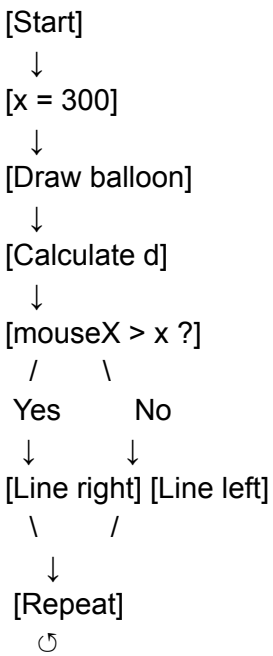
   Calculate distance d

   IF mouseX > x
      Draw line to the right
   ELSE
      Draw line to the left
   END IF

END REPEAT

END
```

Flowchart



Error ที่พบ

หากไม่มี `else` โปรแกรมจะไม่วาดเส้นในกรณีที่เมาส์อยู่ด้านซ้าย

ดังนั้นต้องมี:

```
if mouseX > x:
    ...
else:
    ...
```

Exercise 2 — Draw Positive Signs

โจทย์ให้ทำอะไร

ให้ใช้ `draw_positive()` จาก Lab 3 เพื่อวาดเครื่องหมาย `+` หลายอัน และใช้ loop เพื่อกำหนดจำนวนครั้งที่ต้องวาด โดยโปรแกรมในงานที่ส่งกำหนดจำนวนไว้ที่ตัวแปร `number_of_positive` และใช้ `while` ในการทำซ้ำ

Code

```
number_of_positive = 5

square_size = 60


def setup():
    size(1200, 300)
    rectMode(CENTER)


def draw():
    background(200)

    x = 110
    count = 0

    while count < number_of_positive:
        draw_positive(x, 150, square_size)

        x = x + 220

        count = count + 1


def draw_square(x, y, size):
    fill(200)
    stroke(100)
    rect(x, y, size, size)


def draw_positive(x, y, size):
```

```
    draw_square(x, y, size)
    draw_square(x, y - size, size)
    draw_square(x, y + size, size)
    draw_square(x - size, y, size)
    draw_square(x + size, y, size)
```

อธิบาย

number_of_positive = 5

กำหนดจำนวนเครื่องหมาย + ที่ต้องการวาด

ถ้าต้องการ 3 อัน เปลี่ยนเป็น:

number_of_positive = 3

---

square_size = 60

กำหนดขนาดของแต่ละสี่เหลี่ยม

---

size(1200, 300)

สร้างพื้นที่สำหรับวาด

rectMode(CENTER)

กำหนดให้ตำแหน่งที่ใช้วาดสี่เหลี่ยมเป็นจุดกึ่งกลาง

---

x = 110

กำหนดตำแหน่งเริ่มต้นของ +

count = 0

เริ่มนับจำนวน + ที่วาด

---

while count < number_of_positive:

ทำซ้ำตราบใดที่จำนวนที่วาดยังน้อยกว่าจำนวนที่กำหนด

draw_positive(x, 150, square_size)

วาดเครื่องหมาย +

x = x + 220

เลื่อนตำแหน่งไปทางขวา 220 pixels เพื่อให้มีช่องว่างระหว่าง +

count = count + 1

เพิ่มจำนวนที่วาดขึ้น 1

`draw_positive()`

ตรงกลาง:

draw_square(x, y, size)

ด้านบน:

draw_square(x, y - size, size)

ด้านล่าง:

draw_square(x, y + size, size)

ด้านซ้าย:

draw_square(x - size, y, size)

ด้านขวา:

draw_square(x + size, y, size)

รวมกันเป็นเครื่องหมาย +

Pseudocode

START

Set number_of_positive = 5
Set square_size = 60

Create window

Set x = 110
Set count = 0

WHILE count < number_of_positive

    Draw positive sign at x

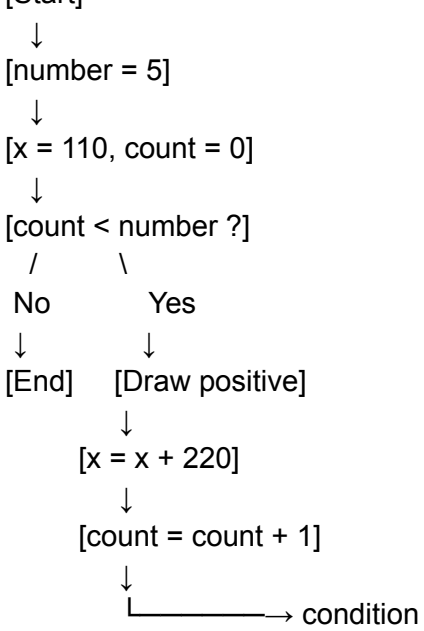
    Move x to the right by 220

    Increase count by 1

END WHILE

END

Flowchart



Error ที่พบ

1. เครื่องหมาย + ติดกัน

เดิมใช้ระยะเลื่อนน้อยเกินไป

แก้โดยใช้:

x = x + 220

เนื่องจาก + มีความกว้างประมาณ 180 pixels จึงเหลือช่องว่างประมาณ 40 pixels

2. จำนวน + ไม่เปลี่ยน

ต้องแก้ที่:

number_of_positive = 5

3. Loop ไม่หยุด

ต้องมี:

count = count + 1

ถ้าลืม update ค่า `count` จะเกิด infinite loop

Exercise 3 — Doraemon Interaction

โจทย์ให้ทำอะไร

ให้สร้างรูป Doraemon จากฟังก์ชันย่อยหลายฟังก์ชัน และสร้าง interaction โดยเมื่อผู้ใช้คลิกเมาส์ โปรแกรมจะตรวจสอบตำแหน่งของเมาส์และทำงานแตกต่างกันด้วย `if-elif-else` นอกจากนี้ส่วนหนวดใช้ `while` แทน `for`

Code

```
import random

doraemon_x = random.randint(300, 700)
doraemon_y = random.randint(300, 700)


def setup():
    size(1000, 1000)
    strokeWeight(4)


def draw():
    background(240, 240, 255)

    draw_doraemon(doraemon_x, doraemon_y)


def draw_doraemon(x, y):
    draw_head(x, y)
    draw_face(x, y)
    draw_eyes(x, y)
    draw_nose(x, y)
    draw_whiskers(x, y)
    draw_mouth(x, y)
    draw_collar(x, y)
    draw_bell(x, y)


def draw_head(x, y):
    fill(0, 150, 255)
    ellipse(x, y, 520, 520)


def draw_face(x, y):
    fill(255)
```

```
    ellipse(x, y + 80, 420, 360)


def draw_eyes(x, y):
    fill(255)
    ellipse(x - 45, y - 110, 110, 130)
    ellipse(x + 45, y - 110, 110, 130)

    fill(0)
    ellipse(x + 35, y - 100, 20, 35)

    fill(255)
    ellipse(x + 30, y - 110, 8, 12)

    stroke(0)
    line(x - 70, y - 100, x - 50, y - 120)
    line(x - 50, y - 120, x - 30, y - 100)


def draw_nose(x, y):
    fill(255, 0, 0)
    ellipse(x, y - 30, 60, 60)

    fill(255)
    ellipse(x - 10, y - 45, 18, 18)

    line(x, y, x, y + 170)


def draw_whiskers(x, y):
    stroke(0)

    i = 0

    while i < 3:
        offset = i * 50

        line(x - 180, y - 20 + offset,
             x - 50, y + 10 + offset)

        line(x + 50, y + 10 + offset,
             x + 180, y - 20 + offset)

        i = i + 1


def draw_mouth(x, y):
    fill(255, 0, 0)
```

```
    arc(x, y + 130, 340, 220, 0, PI)

    line(x - 170, y + 130, x + 170, y + 130)

    fill(255, 140, 0)
    ellipse(x, y + 190, 140, 80)


def draw_collar(x, y):
    fill(255, 0, 0)
    arc(x, y + 260, 380, 60, 0, PI)


def draw_bell(x, y):
    fill(255, 220, 0)
    ellipse(x, y + 315, 90, 90)

    stroke(0)
    line(x - 40, y + 315, x + 40, y + 315)
    line(x, y + 315, x, y + 355)

    fill(0)
    ellipse(x, y + 345, 12, 12)


def mousePressed():
    global doraemon_x, doraemon_y

    distance = dist(mouseX, mouseY, doraemon_x, doraemon_y)

    if distance < 260:
        doraemon_x = random.randint(300, 700)
        doraemon_y = random.randint(300, 700)

    elif mouseX < doraemon_x:
        doraemon_x = doraemon_x + 50

    else:
        doraemon_x = doraemon_x - 50
```

อธิบายโครงสร้าง

```
import random
```

นำเข้า library สำหรับสุ่มตำแหน่ง

```
doraemon_x = random.randint(300, 700)
doraemon_y = random.randint(300, 700)
```

สุ่มตำแหน่งเริ่มต้นของ Doraemon

---

def draw_doraemon(x, y):

เป็นฟังก์ชันหลักสำหรับประกอบส่วนต่าง ๆ ของ Doraemon

จากนั้นเรียก:

draw_head()
draw_face()
draw_eyes()
draw_nose()
draw_whiskers()
draw_mouth()
draw_collar()
draw_bell()

ทำให้โปรแกรมแบ่งเป็น subroutine หลายส่วน

ส่วน `while`

i = 0

กำหนดตัวนับ

while i < 3:

ทำซ้ำ 3 ครั้ง

offset = i * 50

ทำให้หนวดแต่ละเส้นอยู่ห่างกัน

i = i + 1

update ตัวนับ

ส่วน `if-elif-else`

distance = dist(mouseX, mouseY, doraemon_x, doraemon_y)

คำนวณระยะห่างระหว่างเมาส์กับ Doraemon

if distance < 260:

ถ้าคลิกใกล้ Doraemon ให้สุ่มตำแหน่งใหม่

elif mouseX < doraemon_x:

ถ้าคลิกด้านซ้าย ให้ Doraemon เคลื่อนไปทางขวา

else:

กรณีอื่นให้ Doraemon เคลื่อนไปทางซ้าย

Pseudocode

START

Import random

Generate random Doraemon position

Create 1000 x 1000 window

REPEAT
  Clear background
  Draw Doraemon
END REPEAT

WHEN mouse is pressed:

  Calculate distance from mouse to Doraemon

  IF distance < 260
    Generate new random position

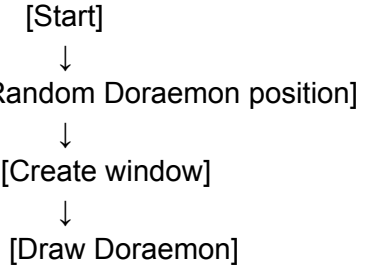
  ELSE IF mouse is left of Doraemon
    Move Doraemon right

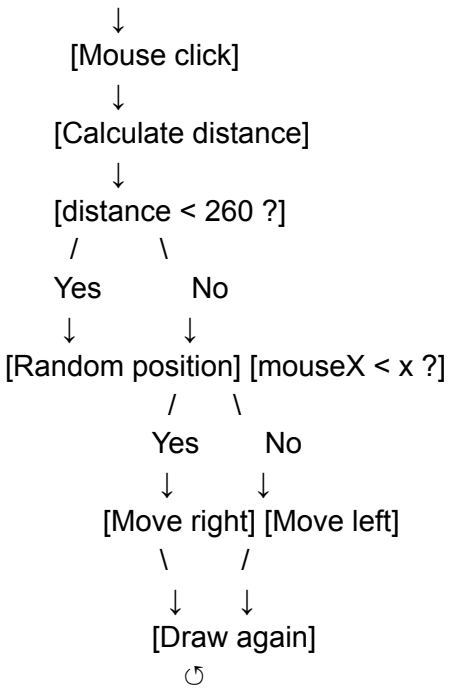
  ELSE
    Move Doraemon left

  END IF

END

Flowchart





Error ที่พบ

1. `for` loop ไม่ตรงตามข้อกำหนด

ใน Lab 3 เดิมใช้:

for i in range(3):

จึงเปลี่ยนเป็น:

```
i = 0
while i < 3:
    ...
    i = i + 1
```

2. Doraemon ขยับไม่ได้

ต้องประกาศ:

global doraemon_x, doraemon_y

เพราะต้องแก้ค่าตัวแปรที่อยู่นอก `mousePressed()`

3. ลืม update `i`

ถ้าไม่มี:

i = i + 1

จะเกิด infinite loop

Part 2: Text Mode

Exercise 4 — Multiplication Table

โจทย์ให้ทำอะไร

ให้ผู้ใช้ป้อนตัวเลข แล้วโปรแกรมแสดงตารางคูณของตัวเลขนั้นตั้งแต่ 1 ถึง 12

ตัวอย่าง ถ้าป้อน 3:

3 x 1 = 3
3 x 2 = 6
...
3 x 12 = 36

Code

```
num = int(input("Enter number: "))


def multiplication_table(num):

    i = 1

    while i <= 12:
        result = num * i
        print(f"{num} x {i} = {result}")
        i += 1


multiplication_table(num)
```

อธิบาย

num = int(input("Enter number: "))

รับตัวเลขจากผู้ใช้และเปลี่ยนเป็น integer

def multiplication_table(num):

สร้าง function สำหรับคำนวณตารางคูณ

i = 1

กำหนดตัวคูณเริ่มต้นเป็น 1

while i <= 12:

ทำซ้ำจนถึงตัวคูณ 12

result = num * i

คำนวณผลคูณ

print(f"{num} x {i} = {result}")

แสดงผลในรูปแบบตารางคูณ

i += 1

เพิ่มตัวคูณทีละ 1

multiplication_table(num)

เรียก function

Pseudocode

START

Input number

Set i = 1

WHILE i <= 12

    result = number × i
    Display number × i = result

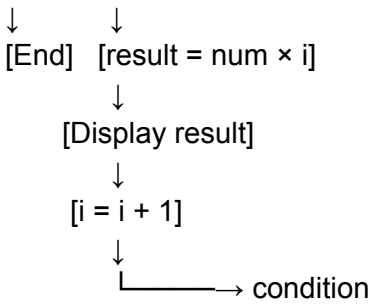
    i = i + 1

END WHILE

END

Flowchart

[Start]
  ↓
[Input num]
  ↓
[i = 1]
  ↓
[i <= 12 ?]
 /   \
No   Yes



Error ที่พบ

1. `i` ไม่เพิ่ม

ถ้าลืม:

i += 1

จะเกิด infinite loop

2. รับ input เป็น string

ถ้าเขียน:

num = input(...)

อาจทำให้การคำนวณไม่เป็นไปตามที่ต้องการ จึงใช้:

int(input(...))

Exercise 5 — Perfect Number

โจทย์ให้ทำอะไร

ให้ผู้ใช้ป้อนตัวเลข แล้วโปรแกรมตรวจสอบว่าตัวเลขนั้นเป็น **Perfect Number** หรือไม่ พร้อมแสดงตัวหารและขั้นตอนการคำนวณ

ตัวอย่าง:

Enter a number: 28

Divisors: 1 + 2 + 4 + 7 + 14
Calculation: 1 + 2 + 4 + 7 + 14 = 28
28 is a perfect number.

Code

```
def perfect_number(num):
    divisor = 1
```

```
    total = 0
    cal = ""

    while divisor < num:
        if num % divisor == 0:
            total = total + divisor

            if cal == "":
                cal = str(divisor)
            else:
                cal = cal + " + " + str(divisor)

        divisor = divisor + 1

    print("Divisors:", cal)
    print("Calculation:", cal, "=", total)

    if total == num:
        print(num, "is a perfect number.")
    else:
        print(num, "is not a perfect number.")


num = int(input("Enter a number: "))

perfect_number(num)
```

อธิบายทีละส่วน

def perfect_number(num):

สร้าง function สำหรับตรวจสอบ Perfect Number

divisor = 1

เริ่มตรวจสอบตัวหารจาก 1

total = 0

เก็บผลรวมของตัวหาร

cal = ""

สร้าง string สำหรับแสดงขั้นตอนการคำนวณ

---

while divisor < num:

ตรวจสอบตัวเลขตั้งแต่ 1 จนถึง `num - 1`

เราไม่รวมตัวมันเอง เพราะนิยามของ Perfect Number ไม่รวมตัวเลขนั้นเอง

---

if num % divisor == 0:

ตรวจสอบว่า `divisor` หาร `num` ลงตัวหรือไม่

ถ้า remainder เป็น 0 แสดงว่าเป็นตัวหาร

---

total = total + divisor

นำตัวหารมาบวกในผลรวม

---

if cal == "":

ตรวจสอบว่ายังไม่มีตัวเลขในข้อความคำนวณหรือไม่

ถ้ายังไม่มี:

cal = str(divisor)

ใส่ตัวเลขตัวแรก

ถ้ามีแล้ว:

cal = cal + " + " + str(divisor)

เพิ่ม + และตัวหารตัวใหม่

---

divisor = divisor + 1

ไปตรวจสอบตัวหารตัวถัดไป

หลังจบ loop:

print("Divisors:", cal)

แสดงตัวหาร

print("Calculation:", cal, "=", total)

แสดงวิธีคำนวณ

จากนั้น:

if total == num:

ตรวจสอบว่าผลรวมเท่ากับตัวเลขที่ป้อนหรือไม่

ถ้าเท่ากัน:

print(num, "is a perfect number.")

ถ้าไม่เท่ากัน:

print(num, "is not a perfect number.")

Pseudocode

```
START

Input num

Set divisor = 1
Set total = 0
Set calculation = empty

WHILE divisor < num

   IF num MOD divisor = 0

      Add divisor to total

      IF calculation is empty
         Add divisor to calculation
      ELSE
         Add " + divisor" to calculation
      END IF

   END IF

   divisor = divisor + 1

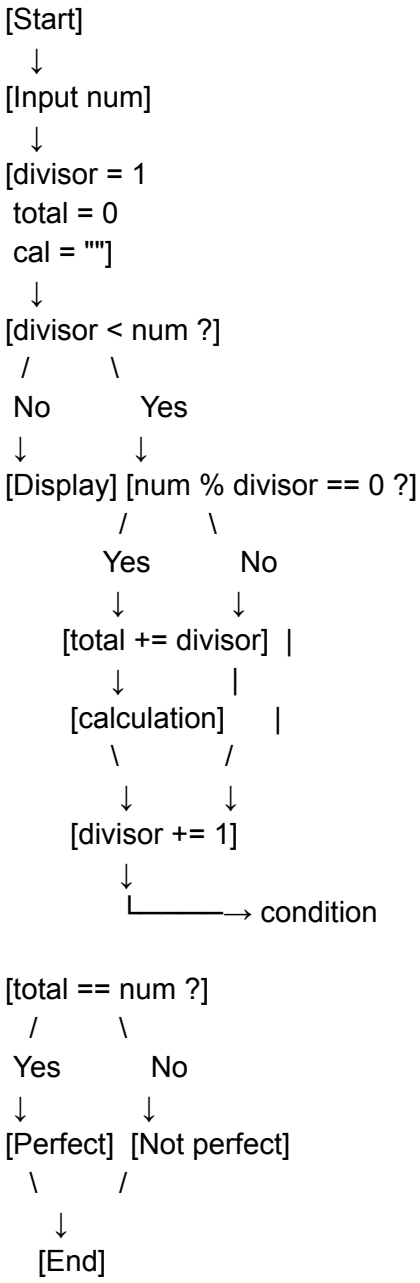
END WHILE

Display divisors
Display calculation and total

IF total = num
   Display "Perfect number"
ELSE
   Display "Not a perfect number"
END IF

END
```

Flowchart



Error ที่พบ

1. รวมตัวเลขตัวเองเข้าไป

เช่นตรวจ 6 ถ้าตรวจถึง 6 จะไม่ถูกต้อง เพราะนิยามให้ไม่นับตัวมันเอง

จึงใช้:

while divisor < num:

แทน:

while divisor <= num:

2. ลืมเพิ่ม `divisor`

ต้องมี:

divisor = divisor + 1

ไม่เช่นนั้น loop จะไม่จบ

3. แสดง calculation ไม่ถูกต้อง

ต้องแยกกรณีตัวแรกกับตัวถัดไป เพื่อไม่ให้ output กลายเป็น:

+ 1 + 2 + 4

Exercise 6 — Number Guessing Game

โจทย์ให้ทำอะไร

ให้โปรแกรมสุ่มตัวเลขตั้งแต่ 1–100 จากนั้นให้ผู้ใช้ทายตัวเลข ถ้าทายผิดให้บอกว่าสูงหรือต่ำเกินไป และให้ทายใหม่จนกว่าจะถูก ผู้ใช้สามารถป้อน `0` เพื่อยกเลิกและดูคำตอบ

Code

```
import random

num = random.randint(1, 100)

while True:
    guess = int(input("Guess the number (1-100) or enter 0 to quit: "))

    if guess == 0:
        print(f"The number was: {num}. Thanks for playing!")
        break

    elif guess < num:
        print("Too low! Try again.")

    elif guess > num:
        print("Too high! Try again.")

    else:
        print(f"Congratulations! You've guessed the number {num} correctly!")
        break
```

อธิบาย

import random

นำเข้า module สำหรับสุ่มตัวเลข

num = random.randint(1, 100)

สุ่มเลขระหว่าง 1–100

while True:

สร้าง loop ที่ทำงานต่อไปเรื่อย ๆ จนกว่าจะเจอ `break`

guess = int(input(...))

รับตัวเลขที่ผู้ใช้ทาย

if guess == 0:

ถ้าผู้ใช้ป้อน 0 ให้จบเกม

break

หยุด loop

elif guess < num:

ถ้าตัวเลขที่เดาต่ำกว่าคำตอบ

elif guess > num:

ถ้าตัวเลขที่เดาสูงกว่าคำตอบ

else:

หมายถึงเดาถูก

Pseudocode

```
START

Generate random number from 1 to 100

WHILE True

    Input guess

    IF guess = 0
        Display answer
        BREAK

    ELSE IF guess < answer
        Display "Too low"
```

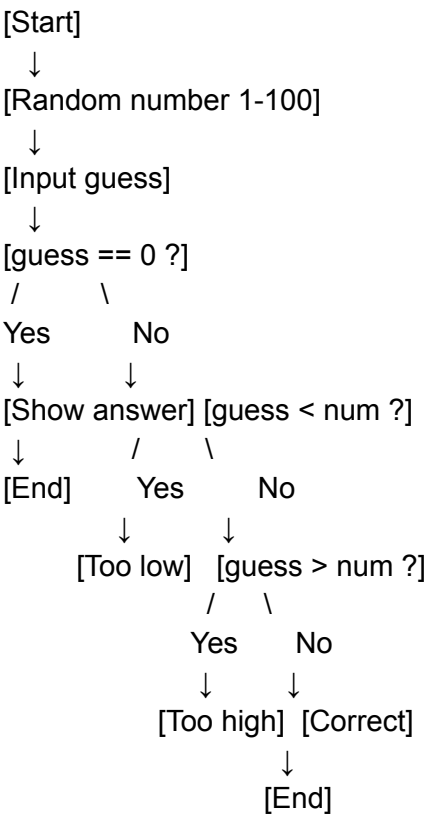
```
    ELSE IF guess > answer
        Display "Too high"

    ELSE
        Display "Correct"
        BREAK

END WHILE

END
```

Flowchart



Error ที่พบ

1. ไม่มี `break`

ถ้าเดาถูกแล้วไม่มี:

break

โปรแกรมจะถามต่อไปเรื่อย ๆ

2. ใช้ `=` แทน `==`

ผิด:

if guess = 0:

ถูก:

if guess == 0:

Exercise 7 — Durian Weight

โจทย์ให้ทำอะไร

ให้รับน้ำหนักทุเรียนทีละลูก และเมื่อผู้ใช้ป้อน `0` ให้หยุดรับข้อมูล จากนั้นแสดงจำนวนทุเรียนและน้ำหนักเฉลี่ย

Code

```
def calculate_durian_weight():
    count = 0
    total = 0

    durian = int(input("Please input Durian weight: "))

    while durian != 0:
        count = count + 1
        total = total + durian

        durian = int(input("Please input Durian weight: "))

    if count > 0:
        average = total / count
    else:
        average = 0

    print("Total of", count, "Durians")
    print("Average weight is", average)


calculate_durian_weight()
```

อธิบาย

count = 0

เริ่มต้นจำนวนทุเรียนเป็น 0

total = 0

เริ่มต้นน้ำหนักรวมเป็น 0

durian = int(input(...))

รับน้ำหนักทุเรียนลูกแรก

while durian != 0:

ทำซ้ำจนกว่าจะป้อน 0

count = count + 1

เพิ่มจำนวนทุเรียน

total = total + durian

เพิ่มน้ำหนักเข้าไปในผลรวม

durian = int(input(...))

รับน้ำหนักลูกถัดไป

---

if count > 0:

ตรวจสอบว่ามีทุเรียนหรือไม่

average = total / count

คำนวณค่าเฉลี่ย

ถ้าไม่มี:

average = 0

เพื่อป้องกันการหารด้วย 0

Pseudocode

START

Set count = 0
Set total = 0

Input durian weight

WHILE weight != 0

    count = count + 1
    total = total + weight

    Input next weight

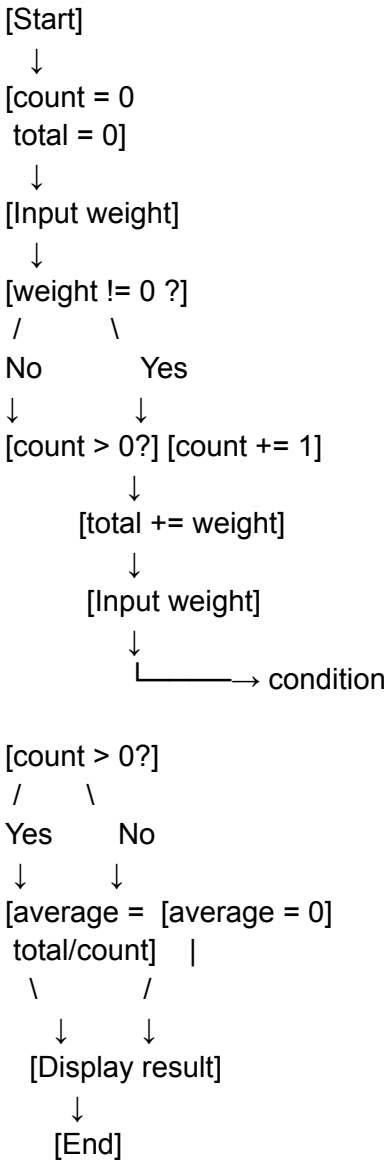
```
END WHILE

IF count > 0
   average = total / count
ELSE
   average = 0
END IF

Display count
Display average

END
```

Flowchart



Error ที่พบ

1. หารด้วยศูนย์

ถ้าไม่มีทุเรียน:

total / count

จะเกิด `ZeroDivisionError`

จึงแก้ด้วย:

if count > 0:

2. ลืมรับน้ำหนักใหม่ใน loop

ต้องมี:

durian = int(input(...))

อยู่ภายใน loop ไม่เช่นนั้นจะตรวจค่าน้ำหนักเดิมตลอดและ loop ไม่จบ

Exercise 8 — Coin Exchange

โจทย์ใน Lab กำหนดให้แลกเหรียญ **10, 5 และ 1 บาท** และทำงานซ้ำจนกว่าจะป้อน `0`

> **หมายเหตุ:** ในโฟลเดอร์ `Lab4_homework` ที่คุณส่งมา ผมไม่พบไฟล์โปรแกรม coin exchange ดังนั้นส่วนนี้เป็นโค้ดสำหรับทำตามโจทย์ Lab ไม่ใช่โค้ดที่ผมอ้างว่าอยู่ในไฟล์ของคุณ

โจทย์ให้ทำอะไร

ให้ผู้ใช้ใส่จำนวนเงิน แล้วโปรแกรมคำนวณจำนวนเหรียญ 10, 5 และ 1 บาทที่ต้องได้รับ โดยให้โปรแกรมทำซ้ำจนกว่าจะป้อน `0`

Code

```
def exchange_coin(money):

    coin10 = money // 10
    money = money % 10

    coin5 = money // 5
    money = money % 5

    coin1 = money

    print("Coin 10:", coin10)
    print("Coin 5:", coin5)
```

```
    print("Coin 1:", coin1)


money = int(input("Enter money: "))

while money != 0:

    exchange_coin(money)

    money = int(input("Enter money: "))
```

อธิบาย

def exchange_coin(money):

สร้าง function สำหรับคำนวณเหรียญ

coin10 = money // 10

ใช้ floor division เพื่อหาจำนวนเหรียญ 10

เช่น:

28 // 10 = 2
money = money % 10

หาจำนวนเงินที่เหลือหลังจากใช้เหรียญ 10

28 % 10 = 8

จากนั้น:

coin5 = money // 5

หาเหรียญ 5

money = money % 5

หาเงินที่เหลือ

สุดท้าย:

coin1 = money

เงินที่เหลือทั้งหมดเป็นเหรียญ 1 บาท

Pseudocode

START

Input money

```
WHILE money != 0

    coin10 = money DIV 10
    money = money MOD 10

    coin5 = money DIV 5
    money = money MOD 5

    coin1 = money

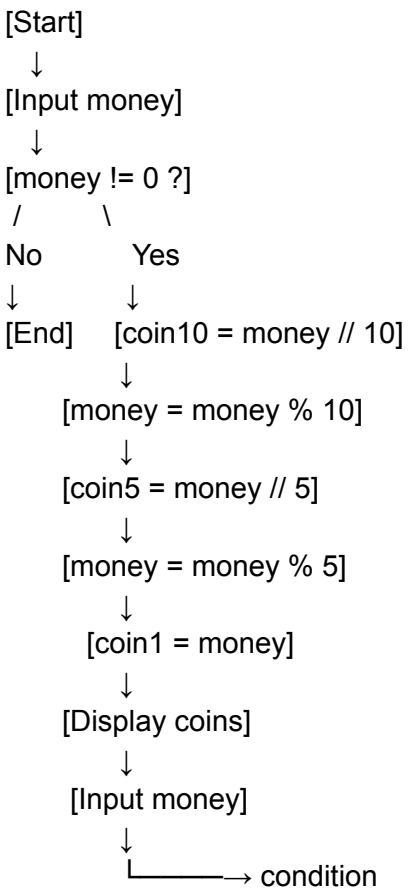
    Display coin10
    Display coin5
    Display coin1

    Input money again

END WHILE

END
```

Flowchart



Error ที่พบ

1. ใช้ `/` แทน `//`

ผิด:

coin10 = money / 10

จะได้ค่าแบบ decimal เช่น `2.8`

ถูก:

coin10 = money // 10

จะได้จำนวนเหรียญเป็นจำนวนเต็ม

2. ลืม `%`

ต้องใช้:

money = money % 10

เพื่อหาเงินที่เหลือหลังจากแลกเหรียญ 10

สรุป Error ที่พบจากการทำ Lab

Error / ปัญหา	สาเหตุ	วิธีแก้
Infinite loop	ลืม update ตัวแปร	เพิ่ม `i += 1` หรือ `count += 1`
Loop ไม่หยุด	เงื่อนไข `while` ไม่สามารถเป็น False	ตรวจ condition และ update
`ZeroDivisionError`	หารด้วย 0	ตรวจ `count > 0` ก่อนหาร
`+` ชิดกัน	ระยะห่างน้อยเกินไป	เพิ่ม `x` เช่น `x += 220`
Function ไม่ทำงาน	ชื่อ function ไม่ตรง	ตรวจชื่อ function
ตัวแปรแก้ค่าไม่ได้ใน Processing	ตัวแปรอยู่นอก function	ใช้ `global`
เดาเลขถูกแต่เกมไม่หยุด	ไม่มี `break`	เพิ่ม `break`
คำนวณเหรียญเป็นทศนิยม	ใช้ `/`	ใช้ `//`

หาตัวหารผิด	รวมตัวเลขตัวเอง	ใช้ `divisor < num`
โปรแกรมรับค่าไม่ได้ตามต้องการ	`input()` ได้ string	ใช้ `int(input(...))`

สรุปแนวคิดที่ใช้ใน Lab 4

จากโปรแกรมทั้งหมด จะเห็นรูปแบบหลักที่ใช้ซ้ำกันคือ

1. Initialise

กำหนดค่าเริ่มต้น

i = 1
count = 0
total = 0

2. Condition

ตรวจสอบว่าจะทำซ้ำหรือไม่

while i <= 12:

หรือ

while count < number_of_positive:

3. Update

เปลี่ยนค่าตัวแปรเพื่อให้ loop สามารถจบได้

i = i + 1

หรือ

count = count + 1

แนวคิด **initialisation → condition → update** เป็นโครงสร้างที่สไลด์สอนโดยตรงในส่วน `while` และเป็นหัวใจสำคัญของ Lab 4.

โครงสร้าง `if-elif-else`

```
if condition:
    # กรณีที่ 1

elif condition:
```

```
    # กรณีที่ 2

else:
    # กรณีอื่น
```

โครงสร้าง `while`

```
initialise

while condition:
    # process
    update
```

โครงสร้าง Function

```
def function_name(parameters):
    # process
```

และถ้าต้องส่งค่ากลับ:

```
def function_name(parameters):
    result = ...
    return result
```

ทั้งหมดนี้ตรงกับแนวทางใน lecture ที่ไล่จาก **variable → conditional → repeated statements → while → initialise/condition/update** ก่อนเข้าสู่ Lab 4.